

Memoria Principale



Obiettivo

- Fornire una descrizione dettagliata dei modi di organizzare l'hardware di memoria
- Discutere le tecniche di gestione della memoria, inclusi il paging e la segmentazione

Background

- Un sistema di elaborazione esegue programmi
- Per essere eseguito un programma deve essere portato (dal disco) in memoria principale e inserito all'interno di un processo
- La memoria principale ed i registri sono la sola memoria a cui la CPU accede direttamente interpretando le istruzioni dei programmi
- Tanti processi devono essere caricati (almeno parzialmente) in memoria contemporaneamente

- La CPU carica le istruzioni indicate dal PC, decodifica ed esegue
- La memoria principale può essere vista come un grande array di bytes con indirizzi
- L'unità di memoria vede solo un flusso di indirizzi + richieste di lettura o indirizzo + dati e richieste di scrittura

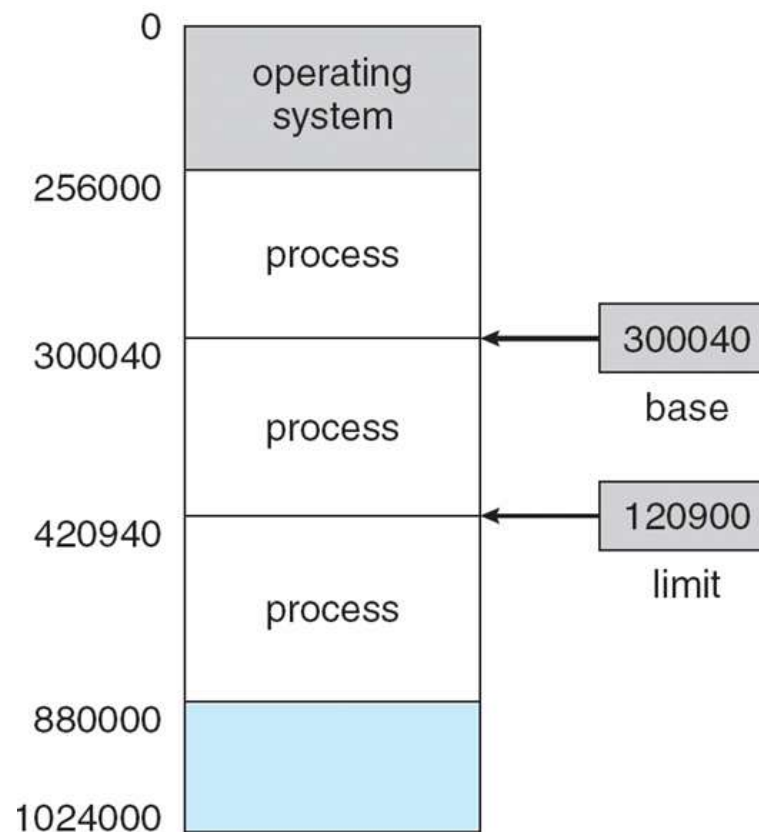
Hardware

- ❑ La memoria principale ed i registri sono la sola memoria a cui la CPU accede direttamente interpretando le istruzioni
- ❑ Supporto hardware per velocizzare l'accesso:
 - ❑ L'accesso ai registri generalmente in un clock della CPU (in alcuni casi più operazioni in un ciclo)
 - ❑ La memoria principale può richiedere molti cicli (memory bus), causando uno stallo in attesa del dato da elaborare
 - ❑ Accesso frequente, troppi stalli
 - ❑ Cache tra la memoria principale e i registri della CPU per accelerare
- ❑ Protezioni hardware:
 - ❑ Necessaria protezione hardware della memoria per garantirne il corretto funzionamento (proteggere sistema e utenti)
 - ❑ Il Sistema non interviene tra CPU e Memoria per non rallentare

Protezione: Registri Base e Limite

- In primo luogo occorre separare i processi in memoria
- Una coppia di registri CPU **base** e **limite** definisce lo spazio degli indirizzi logici per un processo
 - Base il primo indirizzo fisico, limite il range
- La CPU deve controllare ogni accesso in memoria in user mode per verificare che sia tra gli indirizzi base e limite per quel processo

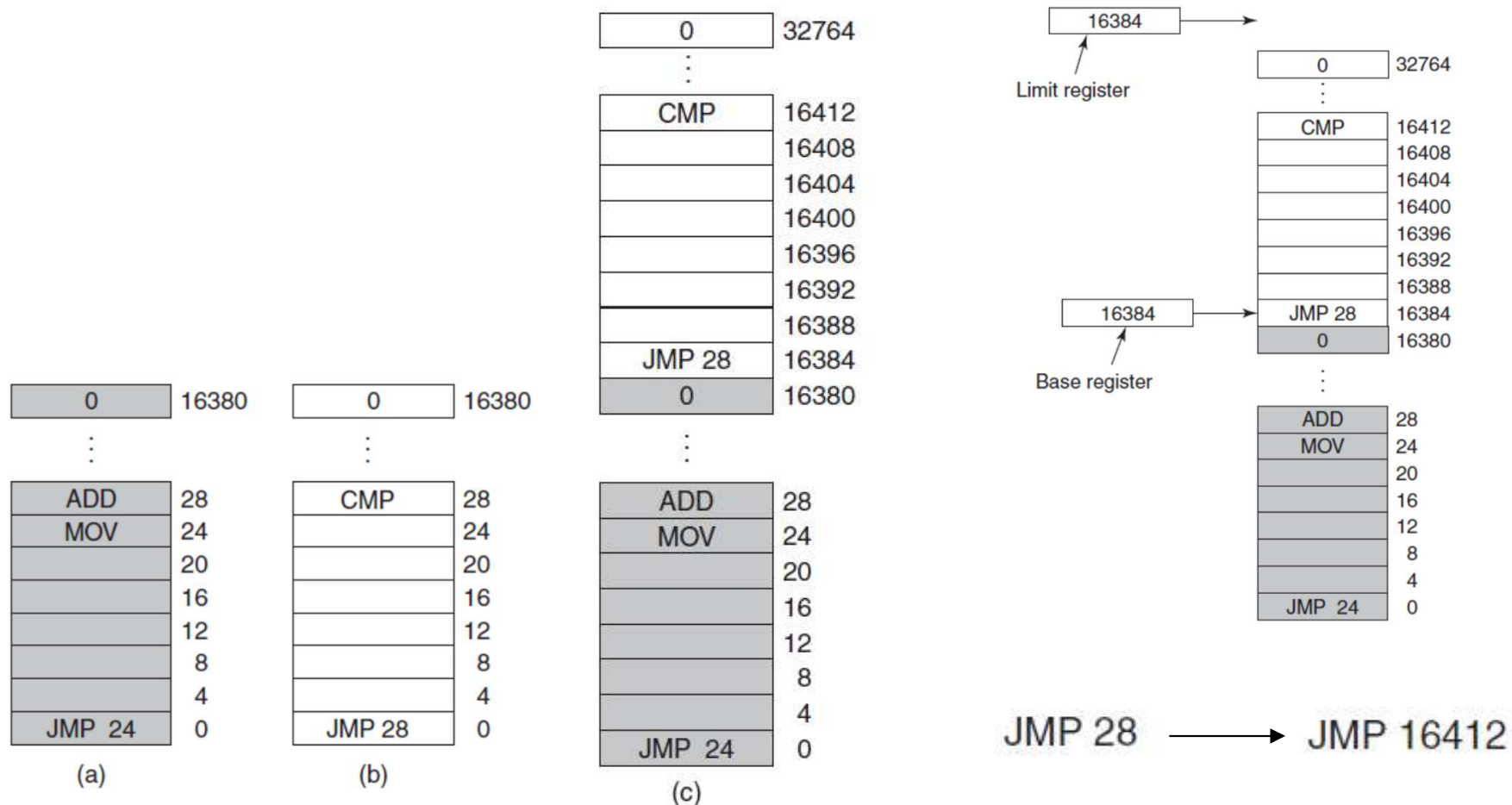
Soluzione CDC 6600
Intel 8088 (non limite),
Intel 80286



Da 300040 a 420939

Protezione: Registri Base e Limite

- In primo luogo occorre separare i processi in memoria
- Una coppia di registri CPU **base** e **limite** definisce lo spazio degli indirizzi logici per un processo
 - Base il primo indirizzo fisico, limite il range

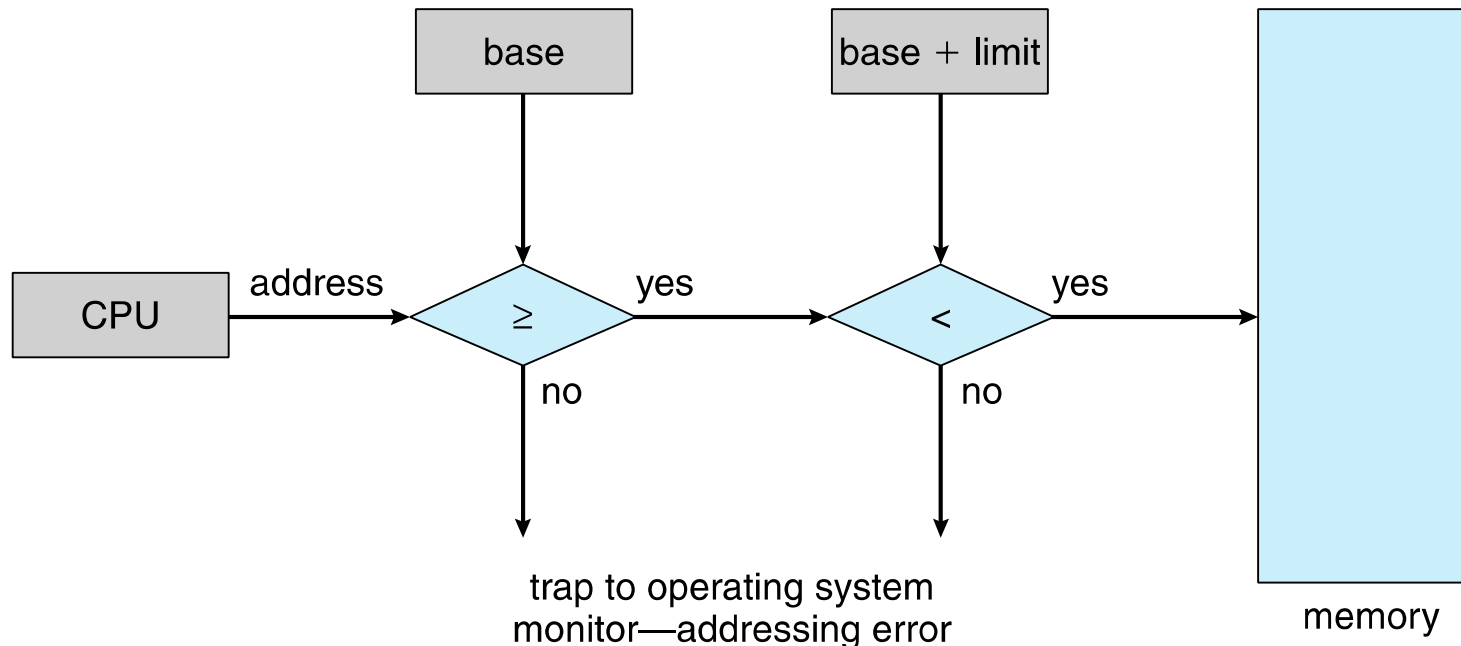


Protezione hardware degli indirizzi

Il controllo di accesso è hardware, se violato l'accesso si genera un **trap** che il Sistema Operativo considera come fatal error

I registri possono essere caricati solo dal SO in modalità privilegiata, quindi solo in kernel mode

In kernel mode il SO ha accesso non ristretto sia alla memoria di SO sia a quella degli utenti

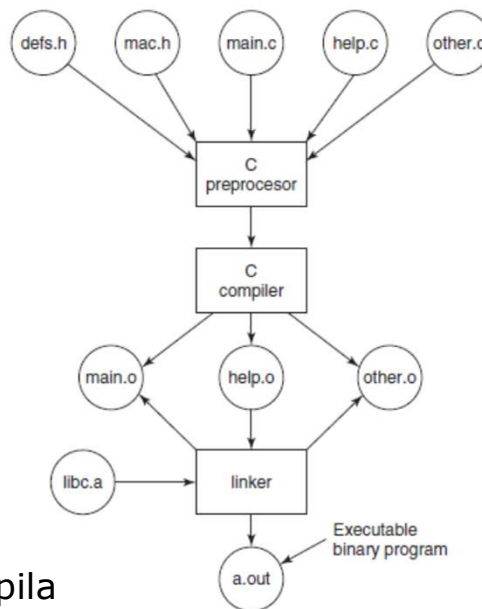


Programmi e Processi

- Programmi su disco
 - file eseguibili pronti per caricamento ed esecuzione
- Programma caricato in memoria
 - La CPU può accedere a dati ed istruzioni
 - Il processo può risiedere in ogni parte della memoria
- Processo termina
 - lo spazio di memoria viene reso disponibile

Processamento Multistep di Programma Utente

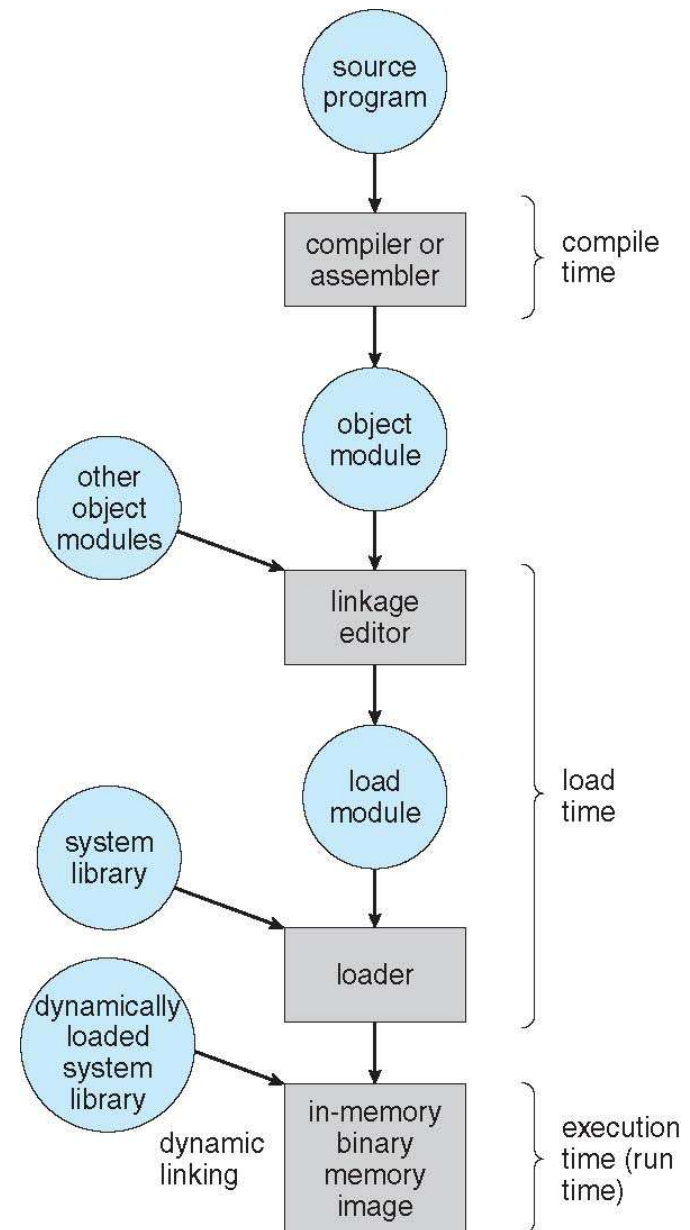
Un programma utente, prima di essere eseguito, deve passare attraverso *varie fasi* (alcune facoltative)



`gcc -E file.c`
preprocessa ma non compila
file.i

`gcc -S file.c`
compila ma non assembla (assembly)
file.s

`gcc -c file.c`
compila e assembla non linka (file oggetto)
file.o



Binding Indirizzi

- Gli indirizzi sono rappresentati in modi diversi nei momenti diversi del ciclo di vita di un programma:
 - Nel codice sorgente sono simbolici (e.g., variabile count)
 - Nel codice compilato gli indirizzi sono associati (**bind**) ad indirizzi rilocabili
 - ▶ es. “14 bytes dall’inizio di questo modulo”
 - Il loader legherà gli indirizzi rilocabili agli indirizzi assoluti
 - ▶ es. 74014
- Il binding rappresenta un collegamento che mappa uno spazio di indirizzi in un altro

Binding di Istruzioni e Dati alla Memoria

- Il binding (degli indirizzi di istruzioni e dati) agli indirizzi di memoria può avvenire in diverse fasi
 - **Tempo di Compilazione:** se al momento della compilazione è già noto dove il processo risiederà in memoria si può generare **codice assoluto**, il codice va quindi ricompilato se la locazione di partenza viene modificata
 - **Tempo di Caricamento:** si genera **codice rilocabile** se la locazione di memoria non è nota a tempo di compilazione; il collegamento finale viene ritardato finché non c'è il caricamento (loader). Se la locazione iniziale per il processo cambia occorre ricaricare il codice.
 - **Tempo di Esecuzione:** se durante l'esecuzione il processo può essere spostato da un segmento di memoria all'altro allora il binding è ritardato fino al run-time
 - ▶ Occorre supporto hardware speciale per mappare gli indirizzi
 - ▶ La maggior parte dei SO usa questo metodo

Indirizzi logici e fisici

- La separazione tra **indirizzi logici** ed **indirizzi fisici** è centrale per la gestione della memoria
 - **Indirizzo logico**
 - ▶ generato dalla CPU; anche chiamato **indirizzo virtuale**
 - **Indirizzo fisico**
 - ▶ indirizzo visto dall'unità di memoria
 - ▶ caricato registro di indirizzi di memoria (memory-address register)
- Indirizzi logici e fisici:
 - **Spazio degli indirizzi logici:** insieme di tutti gli indirizzi logici generati da un programma
 - **Spazio degli indirizzi fisici:** insieme di tutti gli indirizzi fisici che corrispondono ad uno spazio di indirizzi logici

Memory-Management Unit (MMU)

- MMU è un dispositivo hardware che **mappa a run-time** gli indirizzi virtuali in indirizzi fisici
- Diversi schemi di mapping possono essere introdotti
 - Un esempio di schema semplice è basato su registri base e limite
 - In questo caso il registro di base è chiamato **registro di rilocalizzazione**

Unità di gestione della memoria (*memory management unit, **MMU***) svolge l'associazione, nella fase d'esecuzione, tra indirizzi virtuali e indirizzi fisici

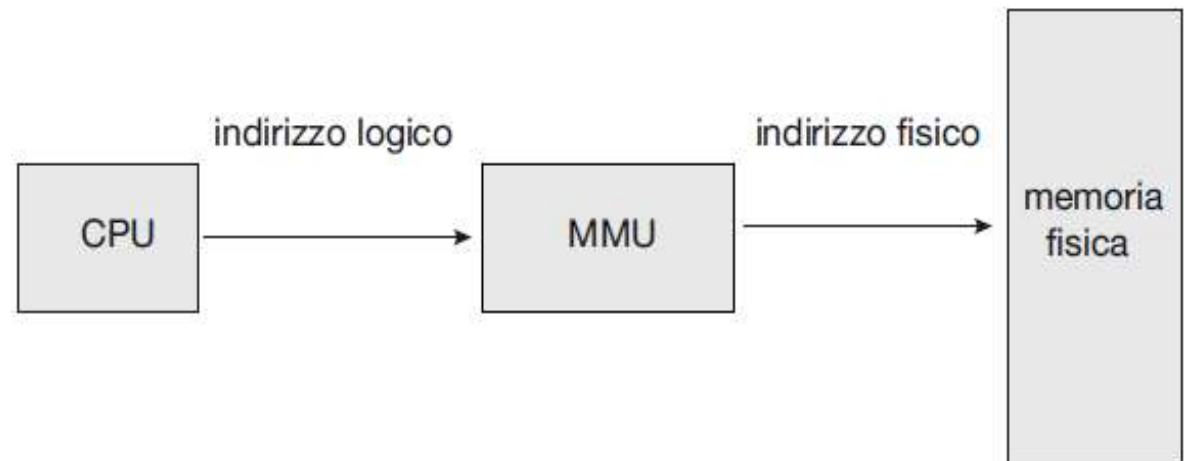
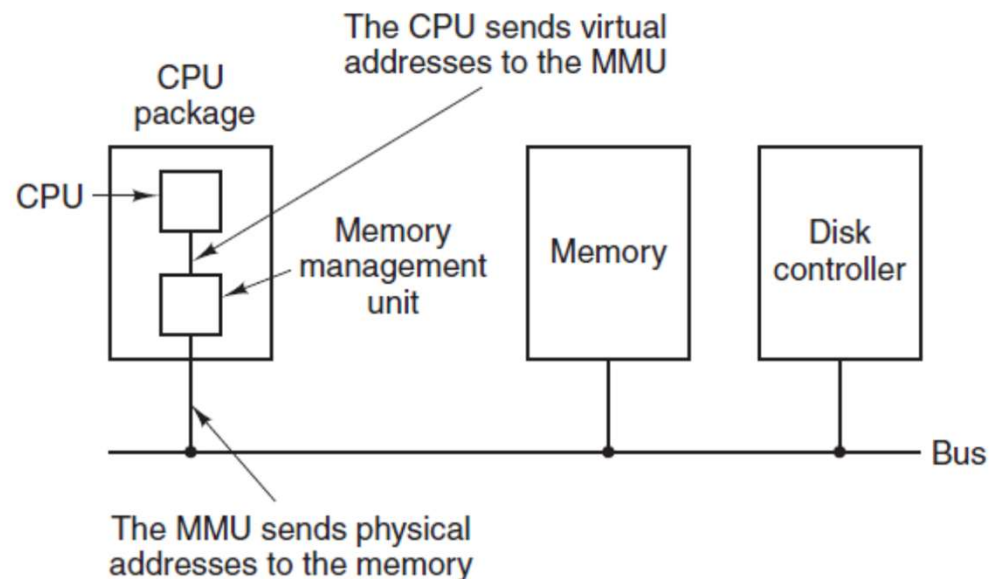


Figura 9.4 Unità di gestione della memoria (MMU).

Memory-Management Unit (MMU)

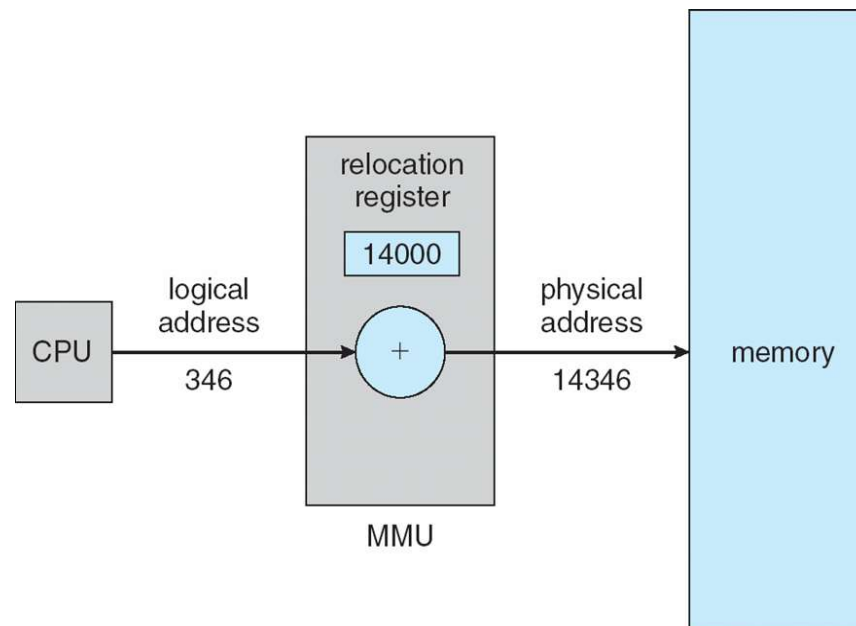
- MMU è un dispositivo hardware che **mappa a run-time** gli indirizzi virtuali in indirizzi fisici
- Diversi schemi di mapping possono essere introdotti
 - Un esempio di schema semplice è basato su registri base e limite
 - In questo caso il registro di base è chiamato **registro di rilocalizzazione**

Unità di gestione della memoria (*memory management unit, MMU*) svolge l'associazione, nella fase d'esecuzione, tra indirizzi virtuali e indirizzi fisici



Memory-Management Unit (MMU)

- MMU è dispositivo hardware che mappa a run time gli indirizzi virtuali in indirizzi fisici
- Diversi metodi sono possibili per fare il mapping
 - Un esempio di schema semplice è basato su registri base e limite:
 - ▶ il valore nel registro di relocation è addizionato all'indirizzo generato dal processo utente
 - ▶ Registro base chiamato ora **relocation register**



- ▶ Es. CPU considera 346 (logico) che viene mappato su 14346
- ▶ MS-DOS su Intel 80x86 usava 4 relocation registers

Memory-Management Unit (MMU)

- MMU è dispositivo hardware che mappa a run-time gli indirizzi virtuali in indirizzi fisici
- Il processo utente vede solo gli indirizzi logici, non vede mai gli indirizzi fisici reali
 - Es. Può gestire un puntatore, manipolarlo etc. rimanendo nello spazio logico
 - Il binding a tempo di esecuzione occorre solo quando si accede ad una locazione di memoria fisica. In questo caso l'hardware per il mapping in memoria converte l'indirizzo virtuale in indirizzo fisico
 - Il processo utente lavora nello spazio di indirizzi logici (es. intervallo $[0, \text{max}]$) che in memoria saranno mappati in indirizzi fisici (es. $[R, R + \text{max}]$)

Caricamento Dinamico

- ❑ Non tutto il programma deve essere caricato in memoria per essere eseguito
- ❑ Caricamento dinamico (dynamic loading)
- ❑ Miglior utilizzo dello spazio di memoria
 - ❑ Una routine non è caricata finché non è chiamata
 - ❑ routine non usate mai caricate
 - ❑ Tutte le routine tenute su disco in formato rilocabile
- ❑ Utile soprattutto quando una grande quantità di codice necessaria per gestire casi non frequenti

Caricamento Dinamico

❑ Non tutto il programma deve essere caricato per essere eseguito

❑ Es. API in Linux and Solaris

❑ include header <dlfcn.h>

❑ dlopen()

- ▶ carica lo shared object dinamico (es. Libreria matematica)

❑ dlsym()

- ▶ ottiene l'indirizzo di un simbolo definite nello shared object

❑ dlclose()

- ▶ decrementa i riferimenti allo shared object se va a zero lo shared object è scaricato

```
#include <stdlib.h>
#include <stdio.h>
#include <dlfcn.h>
#include <gnu/lib-names.h>
```

```
int main(int argc, char **argv) {

    void *handle;
    double (*cosine)(double);
    char *error;

    handle = dlopen (LIBM_SO, RTLD_LAZY);
    if (!handle) {
        fputs (dlerror(), stderr);
        exit(1);
    }
    cosine = dlsym(handle, "cos");
    if ((error = dlerror()) != NULL) {
        fputs(error, stderr);
        exit(1);
    }

    printf ("%f\n", (*cosine)(2.0));

    dlclose(handle);
}
```

```
gcc -o foo foo.c -ldl
```

Linking Dinamico

- ❑ **Linking statico** – librerie e codice del programma collegate dal loader nell'immagine binaria del programma in formato eseguibile

- ❑ **Linking dinamico** – linking è postposto fino al tempo di esecuzione
 - ❑ Simile al caricamento dinamico, invece del caricamento posticipato il link
 - ❑ Utilizzato per librerie di sistema (es. C standard library)
 - ❑ Dynamic Linking Libraries (DLLs)
 - ▶ evitano il caricamento completo per ogni programma
 - ▶ permettono la condivisione (**shared libraries**) tra processi multipli (usate in Win, Linux, etc.)
 - ▶ semplificano la gestione degli aggiornamenti e delle versioni

 - ❑ Per ogni riferimento alla libreria inserito uno **stub** che indica come localizzare la routine in libreria
 - ▶ Lo stub sostituisce sè stesso con l'indirizzo della routine ed esegue la routine, al prossimo riferimento la routine verrà eseguita direttamente

 - ❑ Richiesto supporto da parte del Sistema Operativo

Linking Dinamico

□ **Linking dinamico** – linking è postposto fino al tempo di esecuzione

```
int add(int a, int b);  
int sub(int a, int b);
```

mymath.h

```
// add.c  
int add(int a, int b){ return (a+b); }  
// sub.c  
int sub(int a, int b){ return (a-b); }
```

add.c,
sub.c

```
#include <mymath.h>  
#include <stdio.h>  
#include <stdlib.h>  
  
int main()  
{  
    int x, y;  
    printf("Enter two numbers\n");  
    scanf("%d%d",&x,&y);  
  
    printf("\n%d + %d = %d", x, y, add(x, y));  
    printf("\n%d - %d = %d", x, y, sub(x, y));  
  
    return 0;  
}
```

myAddSumTest.c

```
$gcc -fPIC -c add.c sub.c
```

```
$ gcc -shared -o libmymath.so add.o sub.o
```

```
$ gcc -I . -c myAddSumTest.c
```

```
$ gcc -L`pwd` -o myDynamic myAddSum.o -lmymath
```

```
$LD_LIBRARY_PATH=`pwd` ./myDynamic
```

-c opzione che skipa il linking e crea solo file oggetto

-fPIC dice a GCC di generare *position-independent code* (PIC)

crea la shared library libmymath.so

L'opzione -I dice a GCC di cercare gli header (mymath.h) nella directory indicata

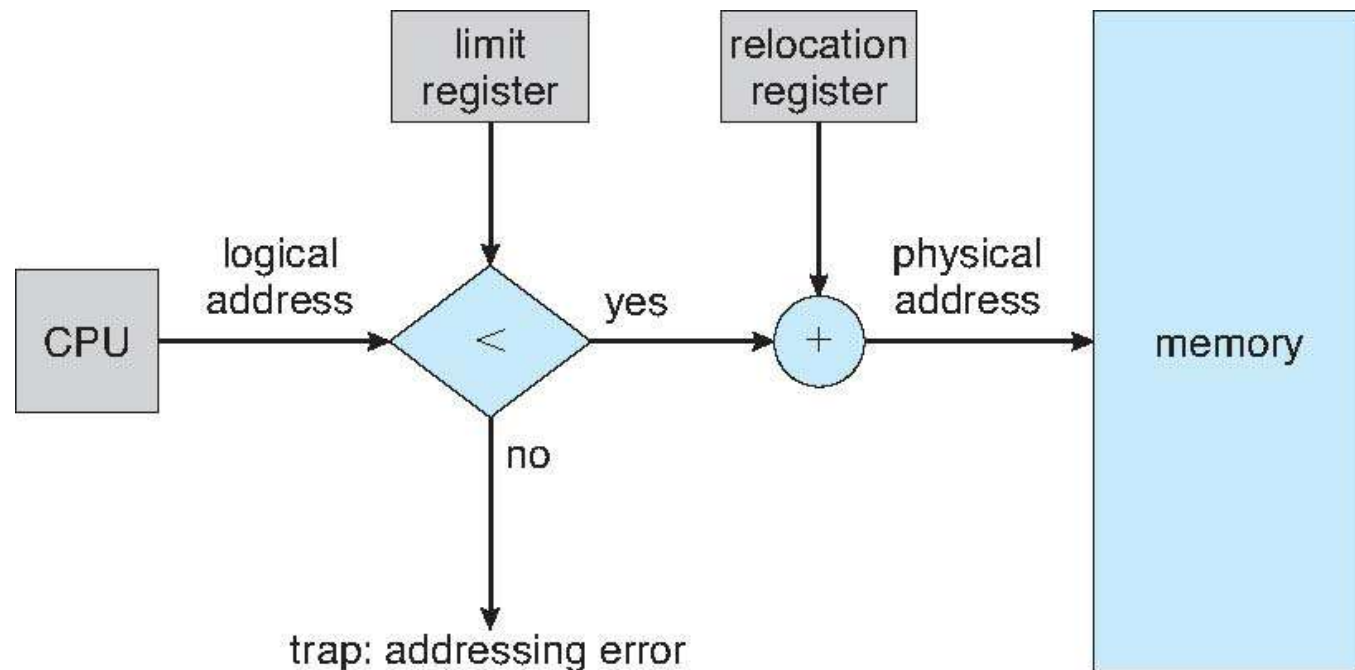
compila indicando la libreria dinamica nella directory corrente

Allocazione Contigua di Memoria

- La memoria principale deve supportare sia i processi di sistema che di utente
- Memoria principale di solito divisa in due **parti**:
 - Sistema Operativo residente tenuto in memoria alta o bassa
 - ▶ Linux e Win in memoria alta
 - Processi utente in memoria bassa o alta
 - ▶ Assumiamo bassa
- Più processi devono essere portati in memoria contemporaneamente
- Allocazione contigua è tra i primi metodi impiegati
 - Processi successivi caricati in sezioni contigue di memoria
 - ▶ Ogni processo contenuto in una singola e contigua sezione della memoria
 - ▶ Necessari meccanismi di protezione della memoria dei processi

Protezione

- ❑ Registro di rilocalizzazione usato per proteggere i processi utente l'uno dall'altro e per impedire il cambiamento di codice e dati di SO
 - ❑ Registro base contiene il valore dell'indirizzo fisico più basso
 - ❑ Registro limite contiene il range di indirizzi logici – ogni indirizzo logico deve essere minore del limite
 - ❑ MMU mappa dinamicamente gli indirizzi logici
 - ❑ Dispatcher carica processo insieme ai registri limite e relocation (context switch)
 - ❑ Anche la dimensione del SO può variare dinamicamente



Allocazione con Partizioni Multiple

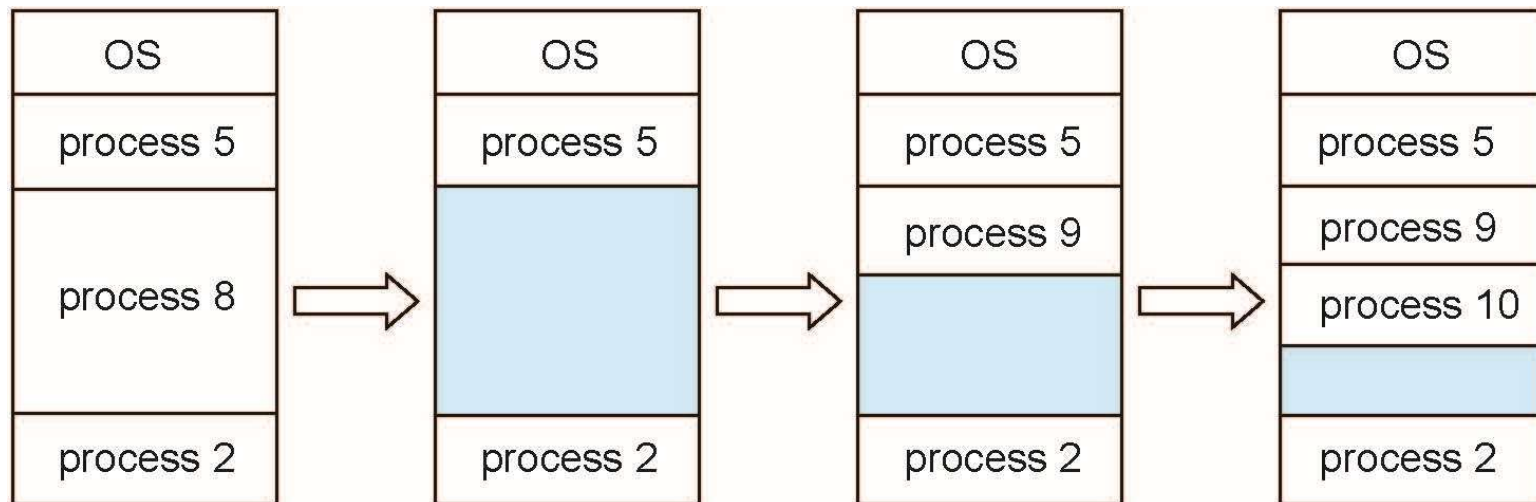
- Partizioni multiple di dimensione variabile
 - Partizioni di **dimensione variabile** dove ciascuna partizione può contenere esattamente un processo
 - Tra i metodi più semplici per l'allocazione della memoria

Allocazione con Partizioni Multiple

- Partizioni multiple di dimensione variabile
 - Ciascuna partizione può contenere esattamente un processo
- Il SO mantiene traccia delle partizioni occupate
 - ▶ **Hole** – blocco di memoria disponibile
 - ▶ Inizialmente tutta la memoria è disponibile (unico hole)
 - ▶ Quando un processo arriva, è allocato in un hole largo abbastanza per gestirlo
 - ▶ I processi uscenti liberano le partizioni e le partizioni libere adiacenti sono combinate in una sola
- Il SO mantiene informazione su:
 - a) partizioni allocate b) partizioni libere (hole)
- Grado di multiprogrammazione limitato dal numero di partizioni disponibili

Allocazione con Partizioni Multiple

- Partizioni multiple di dimensione variabile
 - Allocated gli hole dimensionati sui bisogni del processo



- Se non c'è spazio sufficiente il processo può essere respinto o messo in attesa
- Se il buco è grande viene diviso allocandone una parte e rilasciandone una parte libera
- Ma come scegliere gli hole da allocare?

Problema di Allocazione con Storage Dinamico

Come soddisfare una richiesta di dimensione n da una lista di hole liberi?

- **First-fit:** Alloca il **primo** hole libero che è grande abbastanza
 - Non deve scandire tutta la lista
 - Può scandire dall'inizio o dall'ultima ricerca

- **Best-fit:** Alloca il **più piccolo** hole che è grande abbastanza
 - Deve cercare l'intera lista se non ordinata con la dimensione
 - Produce il più piccolo residuo di hole

- **Worst-fit:** Alloca il **più largo** hole
 - Deve cercare l'intera lista, se non ordinata
 - Produce il più largo residuo di hole

Dalle simulazione si vede che first-fit e best-fit superiori a worst-fit in termini di velocità e utilizzo dello storage

First-fit più veloce, non chiaro invece per quanto riguarda lo storage

Frammentazione

- Problemi di **frammentazione esterna**

- **Frammentazione esterna** – esiste memoria totale per soddisfare una richiesta, ma non è contigua
 - Tanti buchi piccolo non contigui
 - Con first-fit dati N blocchi allocati, $0.5 N$ blocchi sono persi per frammentazione
 - ▶ 1/3 può essere non usabile -> **50-percent rule**

- **Frammentazione interna** – la memoria allocata può essere più grande di quella richiesta
 - La differenza di dimensione è interna ad una partizione, ma non è usata
 - Esempio:
 - ▶ Processo richiede 18462 byte ma buco di 18464, troppo piccolo il residuo per tenerne traccia
 - ▶ Per evitare il problema si divide la memoria in blocchi di dimensione fissa, ma allora si alloca più memoria ogni volta (frammentazione interna)

Frammentazione

- La frammentazione esterna si riduce con la **compattazione**
 - Riassegna i contenuti della memoria per mettere insieme tutta la memoria libera in un blocco di grandi dimensioni
 - La compactazione è possibile se la rilocalizzazione è dinamica ed è fatta a tempo di esecuzione

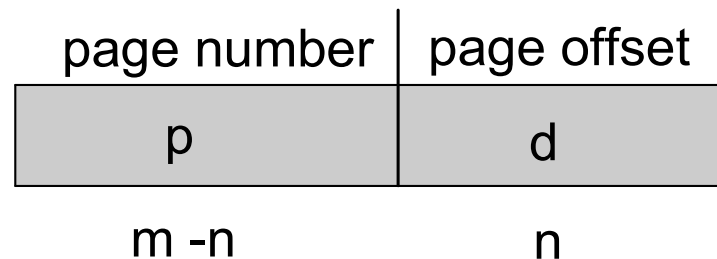
- Un'altra strategia è la **paginazione** che permettere l'allocazione in locazioni non contigue:
 - Il **paging** è la strategia più usata dai moderni SO

Paging

- ❑ Lo spazio dell'indirizzi fisici di un processo può essere non contiguo
 - ❑ Ad un processo è allocata memoria fisica quando è disponibile
 - ❑ Si evita la frammentazione esterna
 - ❑ Si evita il problema di frammenti di memoria con dimensioni variabili
- ❑ Si divide la **memoria fisica** in blocchi di dimensione fissa detti **frame**
 - ❑ Dimensione potenza di 2, tra 4 KB e 2 GB
- ❑ Si divide la **memoria logica** in blocchi della stessa dimensione detti **pagine**
- ❑ Si tiene traccia di tutti i frame liberi
- ❑ Per lanciare un programma di dimensione **N** pagine, si devono trovare **N** frame liberi e caricare il programma
- ❑ Occorre una **page table** per tradurre gli indirizzi logici in fisici
- ❑ Non risolve problemi di frammentazione interna

Schema di Traduzione degli Indirizzi

- L'indirizzo logico generato dalla CPU è diviso in due parti:
 - **Page number** (p) – usato come indice nella **page table** che contiene l'indirizzo base di ogni pagina nella memoria fisica
 - **Page offset** (d) – combinato con l'indirizzo base definisce l'indirizzo di memoria fisica



- Con spazio degli indirizzi logici di dim 2^m e pagine di dim 2^n
- Spazio degli indirizzi logici sparato da quello degli indirizzi fisici

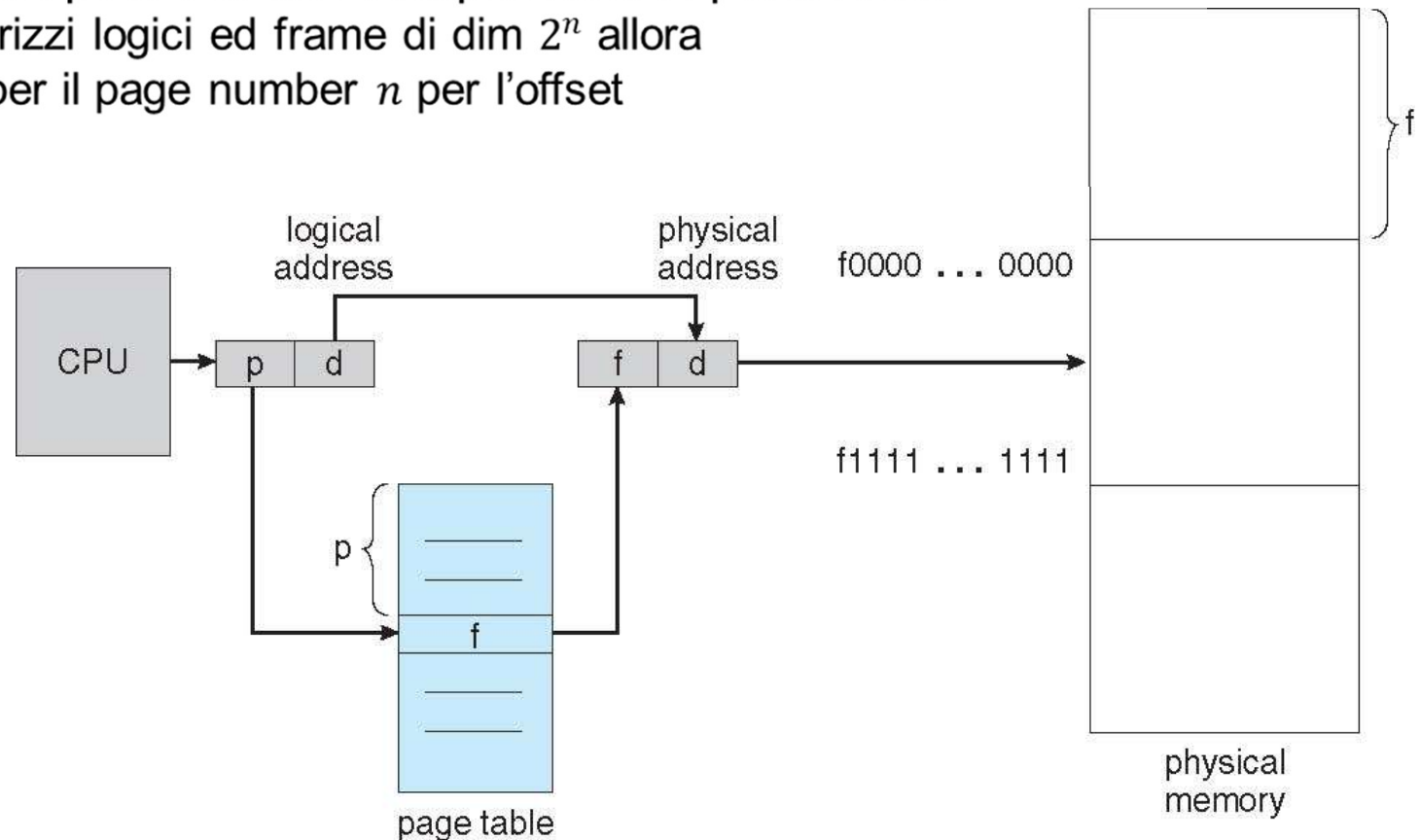
Hardware per il Paging

La MMU esegue i seguenti passi:

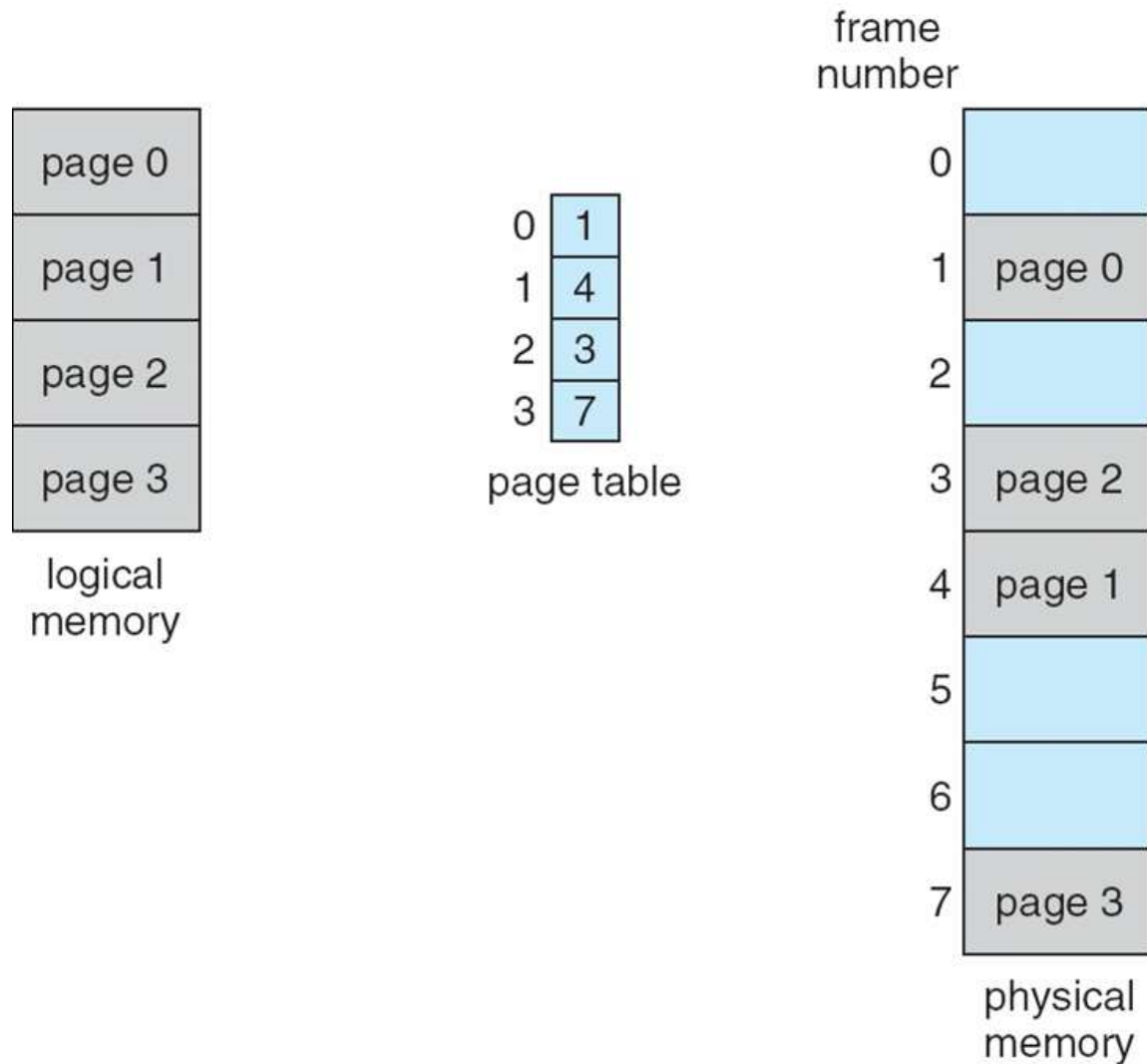
- Estrae in numero p di pagina usandolo come indice
- Estrae il frame number f dalla tabella
- Compone l'indirizzo sostituendo f a p

Le pagine con potenza di 2 semplificano l'operazione:

- Se 2^m indirizzi logici ed frame di dim 2^n allora
- $m - n$ bit per il page number n per l'offset



Modello di Paging di Memoria Logica e Fisica



Esempio Paging

Indirizzo logico 0 = pagina 0, offset 0
La pagina 0 è nel frame 5,
quindi 0 logico si mappa in 20 [= (5 × 4) + 0] fisico

Indirizzo logico 3 (pagina 0, offset 3) mappa
nell'indirizzo fisico 23 [= (5 × 4) + 3]

Indirizzo logico 4 è in page 1, offset 0;
page 1 si mappa nel frame 6.
Quindi 4 logico si mappa nel fisico 24 [= (6 × 4) + 0]

0	a
1	b
2	c
3	d
4	e
5	f
6	g
7	h
8	i
9	j
10	k
11	l
12	m
13	n
14	o
15	p

logical memory

0	5
1	6
2	1
3	2

page table

0	
4	i j k l
8	m n o p
12	
16	
20	a b c d
24	e f g h
28	

physical memory

$n=2$ e $m=4$
memoria 32-byte e pagine 4-byte (8 pagine)

Paging

- Con paging si ha solo frammentazione interna
- Calcolo della frammentazione interna
 - Esempio:
 - ▶ Page size = 2,048 bytes
 - ▶ Process size = 72,766 bytes
 - ▶ 35 pages + 1,086 bytes
 - ▶ Frammentazione interna di $2,048 - 1,086 = 962$ bytes
 - Frammentazione worst case = 1 frame – 1 byte
 - In media frammentazione = $1 / 2$ frame size
 - Quindi piccoli frame desiderabili?
 - ▶ ... ma ogni entry della page table impegna memoria per tracciarla
 - Le dimensioni delle pagine crescono nel tempo
 - ▶ Solaris supporta due dimensioni – 8 KB e 4 MB
 - ▶ Win 10 supporta 4 KB e 2 MB
 - ▶ Linux supporta default (4 KB) e huge pages